

Flow Oriented Programming

-

-

“Nada é permanente, exceto a mudança.” — Heráclito

Objetivo

- Elucidar as regras que regem o desenvolvimento de software
- As regras do desenvolvimento de software são imutáveis

Estrutura

- Introdução
 - Aumento do software queda de produtividade
 - Nos perdemos em conceitos que aumentam drasticamente a complexidade e o custo de manutenção
 - Criar software simples é fácil, difícil é criar um software mantível
 - Manutenção custa 80% do esforço de desenvolvimento
 - A escrita do software é linear
 - Mais difícil de ver panoramicamente
 - Usamos OOP
 - Enxergamos o software orientado a objetos aristotélicos
 - Estruturas estáticas (semânticas)
 - Não enxergamos a ordem, o fluxo de execução
 - Aumento de code base, aumento de programadores não resolve .
 - Redundância de formas para o mesmo objetivo
 - Códigos são redundantes, possuem diferentes formas para se fazer a mesma coisa
 - Formas diferentes de se reproduzir as mesmas estruturas
 - Paradigmas
 - Estruturado
 - Linguagens
 - Frameworks
 - Domínios
 - Reescreve-se as mesmas coisas
 - Programadores
 - Vícios
 - Interpretações
 - Aumento na curva de aprendizagem
 - Mudanças recorrentes na forma de desenvolver software
 - Como programador sentimos que nosso código do ano anterior é ridículo
 - Softwares são pouco intuitivos, diferente de outras engenharias
 - Falta de compreensão do que de fato são feitos os softwares
 - Arquitetura limpa
 - Software não são tão intuitivos como prédios
 - Softwares são feitos de linguagens
 - O objeto do software são símbolos que representam significados
 - Jogos de linguagem
 - O problema da filosofia é o domínio da linguagem
 - Confusão de significados
 - White head
 - Russel
 - Luid
 - Não é possível linguagem única
 - Pensamento mais de húngaro
 - O significado é anterior ao homem
 - O problema do software também é a falta de domínio da linguagem.
 - Dominando a linguagem, domina-se o software, a filosofia e a realidade.
 - Proposta de uma linguagem ubíqua
 - Não compreendemos
 - Diferentes paradigmas
 - Conceitos confusos
 - Conceitos bases que rodeiam o desenvolvimento de software
 - Polimorfismo e Isomorfismo

- Sólido x Abstrato
 - Componentes
 - Encapsulamento
 - Acoplamento e Coesão
 - Imperativo e declarativo
 - Boas práticas
 - GOF
 - SÓLID
 - Muito menos de filosofia
- É possível construir sistemas escaláveis utilizando sempre as mesmas regras que independem de hardware, sistema operacional, linguagem, plataforma ou projeto
 - Linguagem universal
 - Filósofos tentaram alcançar
 - Softwares portáteis para qualquer hardware, so, linguagem ou projeto
 - As regras de cada linguagem, plataforma, projeto não podem impactar na estrutura do código
 - As regras são imutáveis
 - Assim como na filosofia, o movimento não muda, é constante. A ordem dos eventos, Logos, o roteiro é sempre o mesmo, variando apenas os seus objetos processantes e processados.
 - No modelo imperativo
 - Sujeito e objeto
 - O roteiro nunca muda
 - Os elementos que sim
 - Construir software utilizando as mesmas regras independente de
 - Arquitetura de Hardware
 - Arquitetura de Componentes
 - Sistema operacional
 - Paradigma
 - Linguagem
 - Projeto
 - Tudo isso tem um motivo: a filosofia buscava compreender os aspectos imutáveis da realidade, prendendo ao imutável temos bases fortes
- Existe mais filosofia em programação do que se imagina
 - Problema do software
 - OOP
 - Forma vs. Matéria
 - Classe vs. Instância
 - Ganhamos a linguagem ubíqua
 - Não dominamos a linguagem
 - Russel
 - Whitehead
 - Luid
 - Conceitos no desenvolvimento que são refletidos e discutidos há milhares de anos em muitas culturas, religiões e filosofias
 - Desenvolver software está além de software
 - Existem mais detalhes nos softwares que não estávamos enxergando
 - Existe mais filosofia em programação do que se imagina, sendo necessário compreender a linguagem
 - Modelagem de software
 - Filósofos buscavam as formas perfeitas, constantes e eternas
 - Necessita-se compreender filosofia para reproduzir no software
- Aspectos da linguagem
 - Verbo e objeto
- Necessidade de blocos
 - Encaixes como lego
 - Como se n sabemos do que softwares sao feitos?
- Para conhecer as regras e se tornar um bom programador é necessário conhecer primeiro do que os softwares são feitos e como os conceitos filosóficos sustentam as suas bases
 - Para ser um bom programador deve dar mais um passo em direção a linguagem
 - Ajustar as lentes para melhor utilizar as regras
 - Para construir um bom software é necessário conhecer de filosofia
- O que há entre a linguagem, o software, a filosofia, espiritualidade?
- Do que são feitos os softwares?
 - Sistemas são roteiros
 - Programadores são roteiristas
 - Um roteiro possui o quem, quando, para o que e o quê.
 - Quem
 - Arquivo
 - Aplicação
 - Quando
 - Para o quê?
 - O quê?
 - Softwares são roteiros que trabalham com informação
 - Constituídos de estados e movimentos
 - Variáveis e funções
 - A variável guarda informação
 - Herdou da matemática como valor que muda
 - Mudança de conteúdo durante a execução do roteiro
-

- Números
- Textos
- Booleanos
- São caixas que possuem um nome e um conteúdo
- Criar variável
- Atribuir valor a variável
- A função processa a informação
- Tecnologia da informação
- Linguagem
 - Softwares são feitos de linguagem
 - A linguagem tem como objetivo simular a realidade
 - Constituídas de movimentos e estados
 - Linguagem é constituída de mudança de estado
 - Constituídos de símbolos
 - Símbolos são entes
 - Símbolos possuem significado
 - Símbolos representam a diferença
 - Símbolos representam a existência
 - Representa o ser
 - A menor unidade da existência
 - Ente
 - Símbolos pertencem a história da composição
 - Podem ser abstratos ou concretos
 - Abstratos representam espaços
 - Concreto representa os que preenchem
 - Representa mudanças e estados
 - Podem ser polimórficos
 - Símbolos e Endereços
 - Símbolos são endereços
 - Físicos
 - Folhas
 - Preenchimento
 - Polimórfico
 - Matéria
 - Componentes
 - Abstratos
 - Espaço na folha
 - Espaços
 - Isomórfico
 - O espaço é isomórfico
 - Forma
 - Símbolos
 - Abstratos
 - Possuem endereços abstratos
 - Espaço dentro de uma folha
 - Permite ser composto
 - Sólidos
 - Possuem endereço físico
 - Capacidade de preencher
 - Representam
 - Folhas são a matéria do software(composição)
 - Estados
 - Informações
 - Verbos
 -
 - Agentes
 - Comando
 - Interação cliente x servidor
 - O símbolo representa a existência
 - Todo símbolo pertence a uma história
 - Símbolo podem ser abstratos ou sólidos
 - Símbolos sólidos são objetos
 - Objetos são componentes
 - Objeto é um símbolo que pode ser detido de mais de uma forma
 - Objetos são polimórficos
 - Podem possuir mais de uma forma
 - Aristotélico
 - Objetos detidos de características e mudanças
 - Platônico
 - Separação entre objetos agentes e estados
 - Símbolos de linguagem
 - Todo símbolo pertence a uma história
 - Possui significado
 - Menores estruturas da realidade
 - Estruturas diferenciáveis
 - Estrutura fundamental

- Estruturas substituíveis
 - Estruturas de composição
 - Símbolos de texto

- Agentes
 - O agente é o elemento responsável por comandar e/ou executar uma operação.

- Cliente x Servidor
 - Agente servidor comanda a operação
 - Agente cliente e executa a operação
 - Um agente também pode ser servidor e cliente da mesma operação
 - Um agente pode também ser cliente e servidor ao mesmo tempo, possuindo as responsabilidades de comandar e executar a mesma instrução.
 - Relação entre cliente servidor e instrução filha
 - Cada agente possui os seus próprios roteiros
 - Um agente pode conter mais de uma responsabilidade;
 - Mais de um roteiro
 - Agentes processam estados
 - Os agentes são os próprios roteiros.
 - Agentes e roteiros são obrigatoriamente acoplados.
 - Os agentes são os elementos modificadores, enquanto que os estados são os elementos processados.
 - O agente é a origem do movimento
 - O endereço do movimento
 - Precisava?
 - Melhor Legibilidade?

- Roteiros
 - Roteiros são fluxos de execução
 - Roteiros descrevem mudanças de estado
 - Um roteiro descreve um conjunto de instruções
 - Roteiro possui elementos
 - Elementos são substituíveis
 - Composição
 - Um roteiro possui como objetivo transformar um estado inicial em um estado final.
 - Uma instrução pode possuir outras instruções filhas
 - Instruções podem ser ordenadas de forma sequencial
 - Instruções podem ser ordenadas de forma paralela.
 - Uma instrução é comandada por um agente cliente
 - Uma instrução é executada por agente servidor.
 - que serão comandadas por agentes clientes e executadas por agentes servidores
 - Roteiros são escritos utilizando uma linguagem
 - Uma linguagem é constituída por símbolos que representam um ou mais significados
 - Símbolos que representam mais de um significado são denominados como polimórficos
 - Símbolos podem ser concretos ou abstratos
 - Símbolos sólidos representam valores reais
 - Havendo correspondência
 - Símbolos concretos são equivalentes a objetos?
 - Uma instrução possui representações de agente, comando e os processados.
 - Descrevendo o quem, o quê, o quando e o para o quê.

 - Todos os elementos em um roteiro de software são representados por signos de texto, substantivos.
 - Deve haver correspondência entre os endereços internos e externos
 - No software o identificador de cada arquivo é o seu endereço físico
 - Para a execução do roteiro, todos os objetos devem estar presentes
 - Para a execução de um roteiro necessita-se de agentes, roteiros e estados;
 - Roteiros possuem elementos
 - Agentes
 - Objetos
 - Roteiros possuem dois estados
 - Abstrato
 - O roteiro não possui os endereços definidos
 - Os endereços são
 - Personagens
 - Espaços
 - Não preenchidos
 - Sólido
 - Os endereços foram definidos
 - Há correspondência entre os valores do roteiro com os endereços reais do ambiente
 - Roteiros são verbos
 - Verbos são imutáveis
 - No modelo imperativo os verbos necessitam de uma origem
 - Fonte de energia

- Funções
 - Comandos, Mudança
 - Entradas x saídas

- Entradas fêmeas
 - Saídas machos
 - Elemento a processar
 - Processamento
 - Processado
 - Entra porco e sai linguça
- Call Function
 - Acoplamento/Polimorfismo
 - Setup
 - Injeção
 - Execution
 - Entradas
 - Injeção de dependências posicional x por chave
 - Saídas
 - Polimorfismo
 - Representa um roteiro
- Ordem
 - Sequencial
 - Paralelo
- Seleção
 - Estruturas de roteamento
 - Switch
 - If-else
- Repetição
 - For
 - While
 - Do While
 - Recursividade
- Estados
 - Elementos processados
 - Podem ser medidos
 - Possui qualidades
 - Só se pode usar o que está no ambiente
 - Qualidade é o que pode se encaixar
 - Ocupa espaço
 - Sólidos são os seres detidos de qualidade
 - Ocupa, espaços
 - Escopo privado e escopo público
- Como se inicia um roteiro?
 - Cada agente é responsável por comportar e comandar os seus próprios roteiros.
 - Para a execução de um roteiro, necessita-se que estejam presentes no ambiente os agentes, os roteiros e os estados necessários para a operação.
 - deve haver correspondência entre as referências dos símbolos do roteiro com os identificadores dos elementos do ambiente.
 - Há um primeiro agente que inicia a operação.
 - O agente cliente lê o roteiro e comanda, evocando pelos seus devidos identificadores, cada instrução ao agente servidor responsável, compartilhando os elementos necessários para a operação
 - O agente servidor recebe o comando e realiza a operação seguindo o seu roteiro, retornando ao cliente um resultado esperado##
 - A execução do roteiro acaba quando todas as instruções são finalizadas.
- Bons programadores
 - Bons programadores são bons roteiristas
 - Ao invés de cenários reais, controlam máquinas de estados.
 - Distribuir roteiros por responsabilidade
 - Coerência do fluxo de dados
 - Alteração com o menor esforço possível
 - Cenários
 - Substituição de dependências
 - Reutilização de roteiros
 - Mudança de dependências
 - Substituição de objetos
 - Substituição de ator
 - Separação de responsabilidades
 - Definir acoplamentos
 - Objetos
 - Estados
 - Agentes
 - Funções
 - Evitar decomposições futuras
 - Decomposições geram esforço
 - Reutilização de componentes
 - Mesmo projeto
 - Diferentes projetos
 - Manutenção
 - Múltiplos programadores
 - Tamanho do projeto

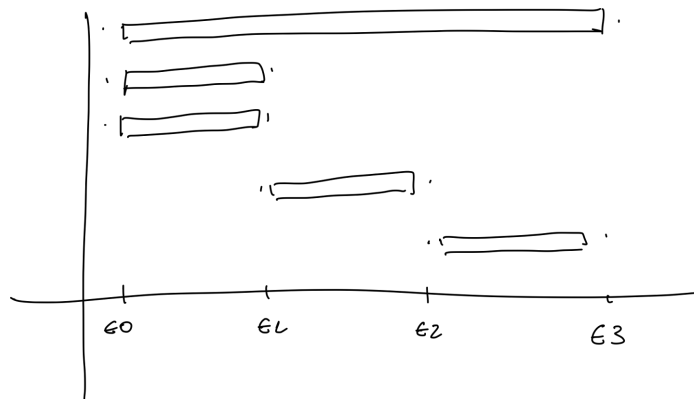
- Uma mudança impacta em todo o projeto
 - Legibilidade
 - Escolha de nomes
- reutilização de roteiros
 - Roteiros independentes a agentes e objetos
 - Dependem de abstrações
 - Entradas de setup
 - Entradas de execução
 - Há acoplamento entre objetos que fazem parte das duas histórias
 - Objetos de estado
 - Separação de
 - Roteiro
 - Agente
 - Estado
 - substituição de elementos
 - **atores**
 - Mudança de biblioteca
 - **objetos**
 - Carta
 - Pizza
 - Caixa
 - Mudança de dependência
 - Dry;
 - Retrabalho
 - Mudança de linguagem
 - Mudança de plataforma
 - Mudança de protocolo
 - Mudança de cenário
 - Risco de incoerência de negócio
- **Divisão de responsabilidades**
 - Definição de acoplamentos
 - Agentes
 - Funções
 - Objetos
 - Qualidades
 - Evitar decomposição
 - Novas funcionalidades
 - Crescimento do sistema
 - Mudam em conjunto ficam em conjunto
 - Evitam acoplamento semântico
 - Apenas statefull devem ser estruturas acopladas?
- **legibilidade**
 - Utilização de símbolos de domínio
 - Divisão de responsabilidades também
 - Diversas equipes trabalhando na mesma base
 - Manutenção
- Bons programadores são bons roteiristas
- Desenvolvem softwares maleáveis as possíveis mudanças;
- Mudança de atores
 - Substituição de bibliotecas
- Compreendem os aspectos da linguagem
- Exercício do roteirista
- Coerência
 - Statefull x Stateless
- Separação de responsabilidades
 - Mudança de responsabilidade por personagem
- Mudança
- Reutilização
- Compreensão
- Pensamento Matemático
- Rodam em estruturas físicas
- Mudanças de atores
- Redistribuição de responsabilidades
- Mudanças de ambiente
- Mudanças de regras de negócio
- Hardwares possuem endereços
- Quando criamos objetos de software, tornamos o verbo um objeto, pois se trata do modelo imperativo
- Filosofia do software
 - Como programadores não compreendemos aspectos da linguagem
 - Polimorfismo
 - Labirinto dos jogos de linguagem
 - Símbolos com múltiplos sentidos
 - Confusão de sentidos
 - Imperativo e declarativo
 - Controle

- Inversão de controle
 - Interfaces bem definidas
 - Protocolo
- Equivalência entre funções e autômatos
 - Alan Turing
- Sujeitos, Verbos e Objetos
- Componente
- Entes
- Símbolos
- Polimorfismo e Isomorfismo
- Polimorfismo de símbolos
 - Estado
 - Verbo
 - Agente
- Acoplamento e Coesão
- Sólido x Abstração
- Dualidade
 - Movimento e Estado
 - Se uma qualidade pode ser percebida é porque há o seu oposto
- Movimento e estado
- Correspondência
- Discutidos no período grego
- Persas
 - Zoroastro
 - Dualidade
 - Tensão
 - Imortalidade da alma
- Matemáticos
 - Formas eternas e perfeitas
 - Ordens iniciáticas
 - Pitágoras
 - Ensinaamentos do Egito
- Gregos
 - Esfera forma perfeita
 - Pré socrático
 - Parmênides
 - Nada muda
 - Heraclito
 - Tudo muda
 - exceto a mudança
 - Socrático
 - Platão
 - Ideias e sentidos
 - Os dois existem, mas separados
 - O que muda
 - O que não muda
 - Demiurgo
 - Aristóteles
 - Separação entre objetos agentes e processados
 - Platão x Aristóteles
 - Substância
 - Forma x matéria
 - Impactos na OOP
 - Seres detidos de características e mudanças
 - Estados e comportamentos
 - Forma e matéria
 - Classe e instância
 - Erro de Aristóteles
 - Unir sujeito e objeto
 - Problemas na OOP
 - Objetos não possuem histórias. Histórias possuem objetos.
 - Platão estava certo
 - Deve haver separação de
 - Agente
 - Ente transformador
 - Objeto
 - Ente transformado
 - Software platônico
 - Separa agente de objeto
 - Os dois existem juntos
 - Substância: Forma e Matéria
- Oriental
 - Taoísmo
 - Dualidade
 - Ying Yang
- Egípcia
-

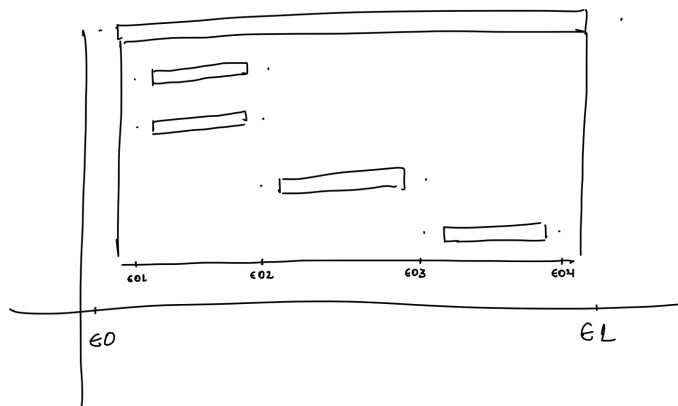
- Corpus hermeticum
 - Pedra
 - Linguagem
 - Hieróglifos
 - Caibalion
 - Correspondência
 - Polaridade
 - Todo é mente
 - Causa efeito
 - Ação e reação
 - Fluxo e Refluxo
 - Gênero
 - Judaísmo
 - José do Egito
 - Símbolos
 - Cristianismo
 - Assim na Terra como no céu
 - São Tomás de Aquino
 - Gnósticos
 - Jesus Cristo gnóstico
 - Ordens iniciativas
 - Maçonaria
 - Rosa cruz
 - Idade média perseguiu quem buscava esse conhecimento
 - Resgatado após a idade média no renascentismo
 - Símbolos de pinturas
 - Iluministas
 - Buscavam a verdade a razão
 - Crítica à metafísica
 - Nietzsche
 - Kant
 - Filosofias modernas
 - Problemas filosóficos decorrem dos labirintos dos jogos de linguagem
 - Linguagem comum é confusa
 - Polimorfismos
 - Redundâncias
 - Linguagem única
 - Uma única forma para se fazer uma coisa
 - Linguagem isomórfica
 - Linguagem ubíqua
 - Roussel
 - White head
 - Luid witnestein
 - Tratados lógicos filosófico
 - Explica a realidade por proposições
 - Jogos de palavras
 - Tudo é linguagem
 - A linguagem é o limite
 - Proposta de continuidade ao trabalho de Russel e whitehead
 - Visão próxima de Yung
 - Símbolos gerais
 - É possível criar uma linguagem unificada
 - Conceitos mal compreendidos da filosofia definem como projetamos software
 - A forma como enxergamos a realidade define a forma como projetamos software
 - A forma como projetamos software pode definir a forma como enxergamos a realidade.
 - A chave da metafísica está no desenvolvimento de software
- Componentes
 - Objetos
 - Formas
 - Tríade
 - O arquivo é o objeto do software
 - Os arquivos são as folhas que representam agentes e objetos de estado
 - Um arquivo pode ser agente, roteiro e estado ao mesmo tempo
 - Polimorfismo é quando um objeto possui mais de uma responsabilidade ao mesmo tempo
 - Árvore de Dependências
 - Todos os elementos que serão necessários para a execução do roteiro
 - Exemplo de árvore de dependências
 - Fronteiras são as leafs
 - Arquivos
 - Os elementos do software são os próprios arquivos;
 - Os arquivos representam agentes, roteiros e estados ao mesmo tempo
 - Imperativo x Declarativo
 - No modelo imperativo o agente comanda a ação
 - No modelo imperativo há uma causa anterior que provoca o movimento
 - No declarativo o movimento decorre do sistema
 - O agente main é o primeiro a comandar

- No modelo imperativo há controle do fluxo
 - Comando
- No modelo declarativo a inversão de controle
 - Inversão de controle
 - Trabalha em cima de contratos bem definidos
 - Interfaces bem definidas
 - Regras compartilhadas
 - Recurso compartilhado?
 - Serve um contrato
 - A estrutura de seleção ou decisão só existe no modelo imperativo
- Polimorfismo e Isomorfismo
 - Objeto Polimorfismo
 - Símbolos que possuem mais de uma responsabilidade
 - Símbolos que estão presentes em mais de uma história
 - Símbolos que possuem mais de um significado
 - Símbolos que possuem mais de uma forma
 - Estrutura tridimensional possui mais de uma forma
 - Objeto Isomorfismo
 - ... uma única responsabilidade
 - ... uma única história
 - ... um único significado
 - Círculo
 - Forma perfeita e eterna
- Tempo de Setup
 - Estado inicial roteiro vazio
 - Estado final roteiro preenchido
 - História da composição
 - Substituição de elementos
 - Definição dos objetos sólidos no roteiro
 - Definição dos elementos do cenário
 - Correspondência entre o mundo real e dos símbolos
 - O resultado é o roteiro com todos os elementos definidos
 - Nessa história você tem o roteiro e os elementos
 - No momento de setup os roteiros são objetos
 - Depois do colapso o roteiro e os elementos de entrada transformam-se em um novo estado
 - História da definição dos elementos sólidos
 - O retorno é o ambiente pronto para iniciar a execução
 - Objetos de composição
 - Quando se cria abstrações cria-se tempo de setup
 - Sem abstração não há composição
 - Deixa as opções abertas
 - Setup do ambiente antes de executar
 - Definições dos elementos sólidos
 - Composição
 - Configuração do ambiente
 - Definição dos elementos necessários para a execução
 - Importações
 - Correspondências
 - Injeção de dependências
 - Entradas são todos os elementos inclusive os agentes
 - Injeção posicional x chave
- Tempo de Execução
 - História da transformação
 - Mudança de estado
 - O retorno é o objeto de estado transformado
 - Objetos agentes e estados
 - Execução
 - Entrada, processamento e saída
 - Entradas são os ingredientes
 - No momento de execução não existem os roteiros?
- Entradas x Saídas
 - Entrada é tudo aquilo que será processado
 - Entrar porco e sai linguça
 - Está na esteira
 - Entrada, processamento e saída
 - Objetos de estado iniciais
 - Objeto de estado final
 - Entradas são as matérias primas do processo
 - A saída é o resultado
 - Agentes são entradas?
 - Parâmetros são entradas?

- Endereços
 - Sólido x Abstrato
 - Sólido: conhece o quem é o quê
 - Abstrato: não conhece o quem, mas conhece o quê
 - O quem é o endereço físico
 - Sólido conhece o quem
- Sólido x Abstrato
 - História da composição
 - Abstratos são os espaços
 - Sólido os preenchimentos
 - Sólido é macho
 - Abstrato é fêmea
 - Definição dos personagens
 - Personagens são espaços a serem preenchidos
- Camadas
 - História da composição
 - Espaço que será preenchidos
 - Torna ato em potência
 - Colapso do movimento para nova matéria
 - Princípio masculino x feminino
- Escopos
 - Escopo global
 - Mesmo contexto
 - Passagem por referência.
 - Mesma sala
 - Ambiente compartilhado
 - Heap memory

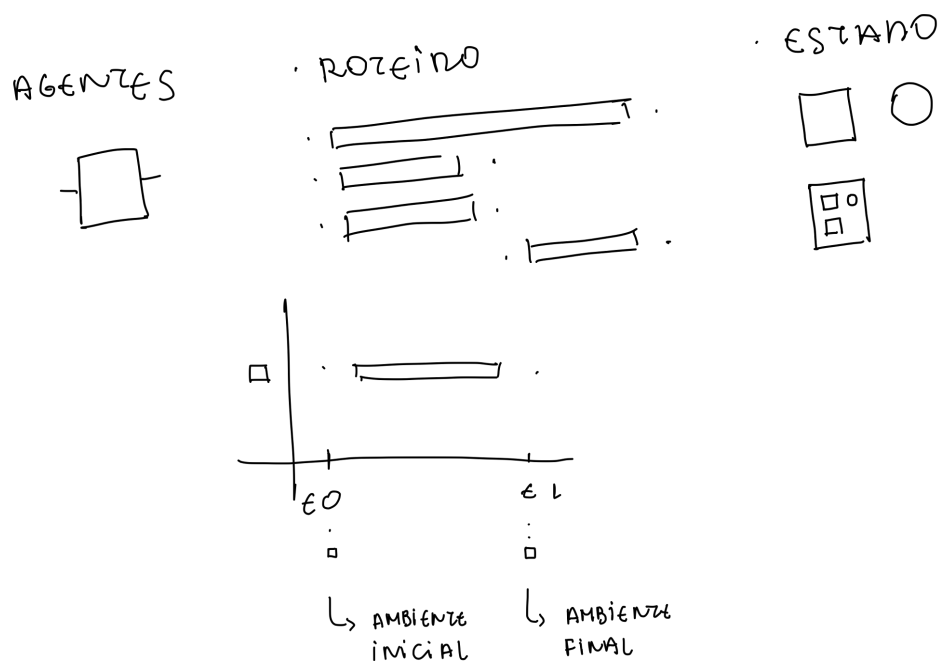


- Escopo privado;
- Diferentes contextos
- Passagem por valor (cópia)
- Sala distinta
- Ambiente dedicado
- Stack memory

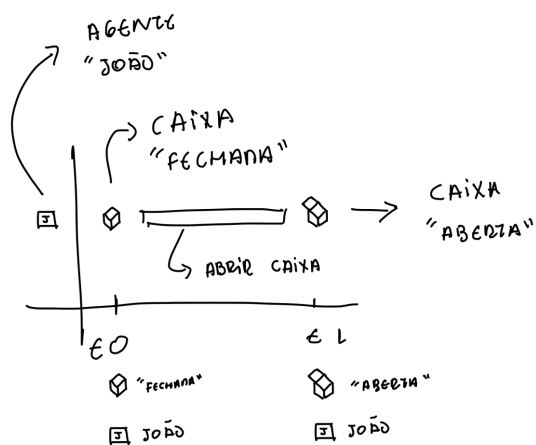


- Statefull x Stateless
 - Agente, roteiro e estado ao mesmo tempo
 - Agente roteiro x estado
- Princípios
 - Responsabilidade única
 - Um sólido para um abstrato
- Pilha de Execução
 - Árvore de execução vira pilha
- Escopo Público x Privado
 - Ambiente privado e ambiente compartilhado
- Imperativo e Declarativo
 - Imperativo define se a ordem de execução
 - Possui fluxo de controle
 - Dado um movimento, um estado
 - Declarativo
 - Inversão de controle
 - Dado um estado, um movimento
- Acoplamento e Coesão
 - Símbolo com mais de uma responsabilidade
 - Coesão quando um símbolo possui uma única responsabilidade
 - Diga não ao acoplamento
 - Essencialismo é a forma perfeita
 - Acoplamento é imperfeito
 - Siamês
- Roteiro x Leaf
 - Business case x execução
- Parâmetro x Referência
 - Cópia
 - Compartilhado
- Estrutura de Dados
- Lazy Load

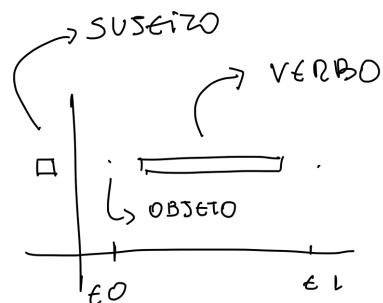
—

[illegible]

O exemplo de abrir uma caixa. O ambiente inicial conta com um agente e uma caixa fechada. Ao realizar a operação de abrir a caixa, o ambiente final conta com a caixa aberta.



Ou seja, um sujeito que executa um verbo a um objeto.



- Os estados são caixas que podem possuir:
 - Verbo;
 - Dado;
 - Outras Caixas (objetos);
- Todos os símbolos possuem nomes
- Algoritmos são shapes

Histórico

- Objetos são estados e agentes
 - Estados por possuírem informação
 - E são agentes por realizarem trabalho
 - Softwares são
 - Roteiros
 - Estados
 - Agentes
 - Diferença entre arquitetura de componentes e de aplicação
 - Componentes são físicos;
 - Atores
 - Aplicação são símbolos;
 - Softwares existem no mundo físico
 - Acoplamento entre roteiro e agente
 - Arquivos são estados, roteiros e agentes
 - Mundo dos valores estados
 - Mundo dos arquivos
 - Mundo dos hardwares
 - Chamadas de funções são
 - Composição
 - Execução
 - Arquitetura de aplicação x componentes
 - N um desenho representando 3 coisas, 3 desenhos representando uma cada
 - **Side effect**
 - Não faz sentido objeto sujeito com um verbo
 - Não faz sentido objeto estado com um estado;
 - Objetos devem ser evitados
 - Acoplamento
 - Mais de uma responsabilidade
 - Dificulta portabilidade
 - Maior o risco
 - Paralelismo de trabalho
 - Menos conflitos
 - Essencialismo
 - Usar objetos apenas em componentes statefull
 - Arquivos que são estado e agente ao mesmo tempo
 - Sujeito e objeto ao mesmo tempo
 - Responsabilidade semântica
 - Unimos responsabilidades que não deveriam estar unidas
 - Aumentamos acoplamento
 - Diminui se portabilidade
 - Arquivos podem ser apenas verbo, estado ou objetos, que são verbos e estados ao mesmo tempo
 - Regras de negócio não devem conhecer regras de hardware ou linguagem
 - Roteiro é correspondente a um verbo e a estado
 - Roteiro = verbo = estado
 - Acoplamento x coesão
 - Verbos não possuem estado?
 - Arquitetura de infraestrutura
 - Arquitetura de aplicação
 - Como estruturamos software sofre forte influência de como enxergamos o mundo e da própria da linguagem
 - O sujeito é o ser que pode ser verbo e objeto ao mesmo tempo
 - Em um roteiro, utiliza-se uma linguagem, onde tudo é representado por símbolos.
 - O agente é a energia capaz de realizar o movimento, o verbo a ordem e o estado a matéria
 - Há distinção entre o modelo imperativo e declarativo
 - Falar sobre a inversão de controles
 - O que é arquitetura e design de aplicações
-
- É de extrema importância compreender os aspectos da linguagem para entender os conceitos que rodeiam a criação de softwares
 - Softwares são feitos de roteiros e nos roteiros existem um conjunto de símbolos de linguagem que representam instruções ordenadas para se chegar a um resultado desejado
 - Um objeto representa tudo aquilo que pode ser substituído em uma história
 - Em um roteiro todos os símbolos que representam agentes, roteiros e estados são considerados objetos por se tratarem de estruturas substituíveis
 - Por se tratar de histórias, os roteiros tornam os próprios roteiros também objetos substituíveis, podendo ser um roteiro um objeto dentro de um outro roteiro
 - A história estar por trás deste evento é a da composição onde existe tanto objeto que compõem como espaço que é composto.
 - É na história da composição que os elementos sólidos e abstratos nascem.
 - O objeto que compõem é o elemento sólido enquanto que o espaço que é composto é o elemento abstrato.

1. Existem muitos problemas no desenvolvimento de software
 - a. Não sabemos o que estamos fazendo
 - b. Presos nas manifestações
 - i. Processos
 - ii. Códigos
 - iii. Não identificamos os padrões
 - c. Inúmeras formas para se fazer a mesma coisa
 - i. Diferentes linguagens
 - ii. Diferentes framework
 - iii. Diferentes paradigmas
 - d. Dificuldade de assimilação com o mundo real
 - i. Orientação a objetos possui bases fracas
 - e. Regras muito difíceis de serem entendidas
 - i. Para cada cenário, os princípios mudam, sendo assim, não são princípios
 - ii. Princípios abstratos de difícil compreensão
 - f. Um padrão que se repete
 - i. A medida que o sistema cresce, a manutenibilidade cai
 - ii. A manutenibilidade caindo, o custo aumenta
 - iii. A manutenibilidade caindo, diminui-se a evolução
 - iv. Problema em cascata até a inviabilidade do projeto
 - g. Em todas as empresas seguem os mesmos problemas
2. Por muito tempo me questionei: “Onde estamos errando?”
 - a. Busca de 15 anos pelas soluções
 - b. Aprofundamento dos estudos
 - i. Funcional
 - ii. Estruturado
 - iii. OOP
 - iv. Clean Architecture
 - c. Estudo dos mais diversos conhecimentos
 - i. Filosofias
 1. Egípcia
 - a. Hermética
 - i. Correspondência
 - ii. Polaridade
 2. Gregas
 - a. Ontologia
 - b. Lógica
 - c. Silogismo
 - i. Dedução
 1. Abstração
 3. Orientais
 - a. Lao Tsé
 - i. Dualidade
 4. Renascentistas
 - a. Iluminismo
 - b. Esoterismo
 5. Modernos
 - a. Carl Jung
 - b. Hegel
 - i. Dialética
 - c. Luid Wittgenstein
 - i. A linguagem é o limite do mundo
 - d. George Peterson
 - ii. Espiritualidade
 1. Judaísmo
 2. Cristianismo
 - iii. Matemática
 1. Lambda
 - iv. Computação
 1. Clean Architecture
 2. Paradigmas
 - a. Procedural;
 - b. Funcional;
 - c. Orientado a Objetos;
 - v. Gestão de processos
 1. Scrum
 2. Kanban
 - vi. Realidade
 1. Existencial;
 - vii. Linguagem
3. Compreender conceitos
 - a. Ontologia

- b. Silogismo
 - c. Dedução
 - d. Indução
 - e. Linguagem
 - f. Abstração
 - 4. Criação de uma hipótese
 - a. Não compreendemos a estrutura do software
 - b. Esta hipótese pode ser uma chave primordial para outras ciências
 - c. Apenas programadores conseguiriam chegar a essa conclusão
 - 5. Desenvolvemos software da forma que enxergamos o mundo
 - a. Enxergamos o mundo orientado a objetos
 - i. O comportamento não pertence a abstração
 - b. Bases fracas não sustentam um software por muito tempo
 - i. Acontece todos os problemas antes referidos
 - 6. Enxergamos o mundo da forma errada
 - a. Comportamentos não pertencem a sujeito
 - i. Comportamento transcende o sujeito
 - 7. Enxergamos software da forma errada
 - a. Acoplamos funções e dados em uma única entidade
 - i. Problema da abstração comportamental
 - 1. Comportamentos não derivam de objetos, objetos derivam de comportamentos
 - 8. Devemos enxergar orientado a história
 - a. Uma história representa uma mudança de estado
 - b. Tudo que existe é compreendido como mudança de estado
 - c. A linguagem é uma ferramenta de descrição de mudanças de estado
 - d. Histórias se resumem a verbos e estados
 - e. Comportamentos transcendem os sujeitos
 - 9. O que é um software?
 - a. Roteiro
 - b. Estrutura do software
 - i. Gantt Flow
 - 10. Do que softwares são feitos?
 - a. Histórias
 - b. Mudanças de estado
 - c. Funções e estado
 - 11. Bons desenvolvedores são bons roteiristas
 - a. Os objetos são estados puros
 - 12. O que definem bons roteiristas?
 - a. Prevenção de mudanças
 - i. Mudanças de atores
 - ii. Mudanças de personagens
 - 1. Acoplamento entre dado e comportamento
 - iii. Mudança do enredo
 - 1. Decomposição de função
 - 2. Entraria como reutilização?
 - a. Porta, tampa de garrafa, janela
 - i. Todos possuem histórias compartilhadas
 - b. Reutilização
13. Como solucionar
 - a. Abstração
 - i. Capacidade de passar função como parâmetro
 - 1. Veio do funcional
 - ii. Camadas são possíveis devido a abstração
 - b. Decomposição de comportamentos e dados
 - i. Na oop os objetos possuem características e comportamentos
 - 1. Dado e função acopladas em um ente
 - 2. O sujeito é acoplado ao verbo em um único ente
 - ii. Remoção do sujeito
 - 1. Objetos são acoplados no mundo físico por questões físicas
 - a. Objetos são acúmulos de características
 - 2. Não precisa ser assim no software
 - a. Questões físicas não devem interferir no software
 - b. Questões organizações não devem interferir no software
 - c. Apenas regras de negócio interferem no software
 - d. Todo software é formado pelas mesmas mônadas
 - iii. Verbo e objeto
 - c. Decomposição de função em outras menores
 - i. Reutilização
 - ii. Extensão
14. Coisas que devemos respeitar
 - a. Abstração
 - i. Encapsulamento
 - b. Sequência Cronologia
 - i. Orientado a eventos
 - ii. Não diferencia chamadas sync e async

- iii. **Paralelismo**
 - c. **Singularidade**
 - i. **Uma única forma de se fazer as coisas**
 - ii. **Desacoplamento a nível de serviço em todas as chamadas**
 - d. **Quântico**
 - i. **Dados são funções até que sejam medidos**
- 15. **Este método pode ser utilizado em todas as ciências**
 - A tríade nasce do terceiro imperativo
 - Objeto que pode ser comandante
 - Há um comando entre cliente e servidor
 - O endereço é a folha
 - Correspondência entre um roteiro preenchido e um novo estado
 - Retornos sempre são abstratos?
 - O como está encapsulado no servidor;
 - O próprio roteiro
 - Imperativo e declarativo são equivalentes
 - Escrever o artigo
 - Escrever o código que cria o artigo
 - Separação de entes
 - Agentes
 - Objetos
 - Padrão entrada, processamento e saída
 - Desacoplamento a nível de serviço em todas as chamadas
 - Independente de protocolo de comunicação
 - Linguagens event based
 - Javascript
 - Acoplamento entre chamada e injeção de dependências
 - Agentes e são estruturas presentes em roteiros
 - Objetos são estruturas presentes em roteiros
 - Injeção posicional x chave
 - Um roteiro puro não possui sólidos
 - Todo software precisa ter uma função
 - O comando é a palavra ação
 - Colapso do movimento
 - Correspondência entre roteiro sólido e resultado
 - Transforma ato em potência
 - Estado inicial em final
 - Correspondência de roteiro sólido para resultado
 - Correspondência entre os seres permite memoização.
 - Se você remove o conceito de camadas acoplado com o desacoplamento a nível de serviço, se tudo vira desacoplamento a nível de serviço, logo, tudo vira camada
 - O que diferencia a entrada de injeção de dependências é que um trata a nível de setup e o outro de execução.
 - Problema da linguagem
 - Discussões filosóficas como Luid
 - Polimorfismo
 - Símbolos com mais de um significado
 - Gera uma confusão de sentidos
 - Objetos de substituição
 - Objeto da linguagem
 - O ideal é que metadados não estejam dentro da estrutura do software
 - Herança apenas de estados
 - Separar elemento de objeto
 - Não misturar verbo com estado
 - Array?
 - `array.push(element)`
 - `push(array, element)`
 - Objetos do software
 - Hardware
 - Arquivo
 - Símbolos
 - Jogos de linguagem
 - Sólido, x abstrato
 - Diferença de import para call function
 - Composição manual
 - Define se na mão o endereço
 - Composição dinâmica
 - Endereços abstratos
 - No software, o quem é o endereço
 - Local x remoto
 - O símbolo está dentro de um arquivo
 - A base do símbolo é o arquivo
 - `/path.metodo()`
 -

- Layers
- Seria o endereço o componente?
 - Estado ou verbo
 - A folha é o endereço
 - O arquivo é a camada
 - Em uma camada pode haver espaços
 - Conectores públicos e privados ?
- A palavra componente
 - Componente que compõe
- O roteiro é o Logos
- Qual a relação entre
 - Forma e Matéria
 - História do encaixe
 - Componente
 - Espaço
 - Movimento e Estado
 - História do processo
 - Modifica
 - Modificado
- Para toda qualidade deve haver a não qualidade
- Toda qualidade pertence a um verbo
- Séria Aristóteles o próprio Demiurgo?
 - O efeito da substância esquece o que é eterno?
- Separação dos seres
 - Aristóteles matou o mundo perfeito
 - Platão separava os dois mundos
- Polimorfismo da substância
 - Substância
 - Símbolo
 - Ente
 - Sólido
 - Parado
 - Estar
 - Visível
 - Estado
 - Objeto
 - Material
 - Sólido
 - Preenchimento
 - Macho
 - Ignorância
 - Homem
 - Temporal
 - Mutável
 - Real
 - Único
 - Concreto
 - Imperfeito
 - Abstrato
 - Movimento
- Polimorfismo é a imperfeição
- O ser perfeito é essencial a sua função
- Aristoteles o princípio do terceiro excluído pode ser estar errado
- Focar em funções?
- Olhar o mundo por proposição é errado?
- Devemos separar tudo?
- Usar sempre símbolos diferentes para cada verbo
 - Redefinição de nomes?
-

Toda entrada deve ser compatível com os símbolos?

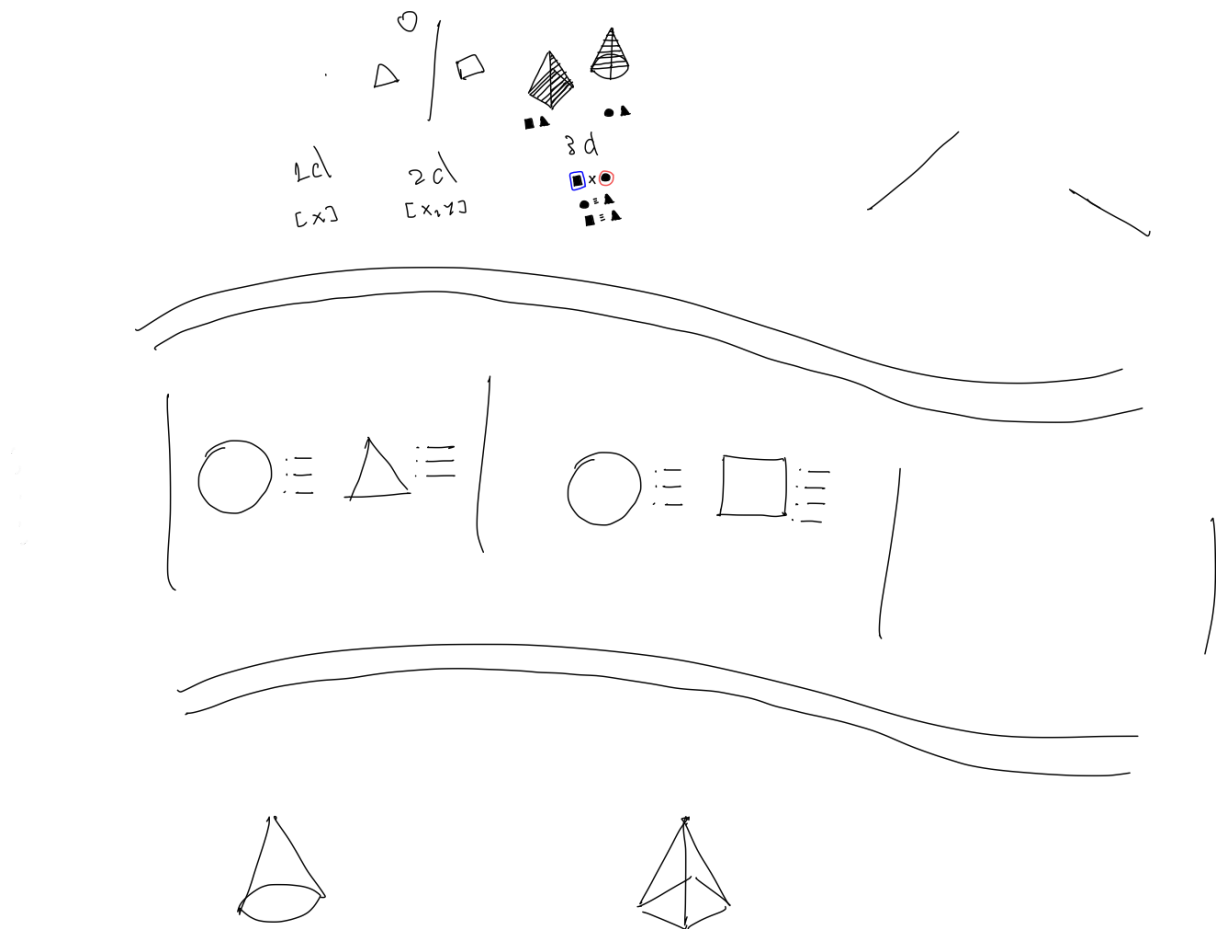
Como fica a relação dos símbolos com cada função e reutilização dos símbolos com abstrações?

Ao usar o backgroundColor você aprisiona o verbo setBackgroundColor ao atributo backgroundColor. Mas, e se o verbo for o mesmo? Exemplo, setValue e o value. Ou setColor e o color.

Não pode haver conflito de nomes de atributos, funções e abstração?

Uma god class, um objeto contendo todos os atributos e funções sem nenhuma incompatibilidade.

Qual a relação entre a Metamorfose e a Abstração?



Objetos polimórficos podem assumir diferentes papéis, contanto que estes não sejam contraditórios, necessitando qualidades opostas.

-
- A abstração pode tornar um ser polimórfico em um ser metamórfico, contanto que não possua qualidades contraditórias.
 - Que não utilizem o mesmo recurso
- Os verbos não possuem qualidades, portanto, não são contraditórios.
- A abstração torna o elemento múltiplo.
- A abstração fatia o elemento
- Os estados possuem qualidades, portanto podem ser contraditórios
- O ser não pode ter dois tamanhos
- Separa-se o verbo do estado.
- O verbo não é contraditório, pois não possui qualidades
- O verbo é a potência a partir do ato
- A capacidade de ser não é ser.
- O ser é o personagem que foi
- A metamorfose resolve um problema, mas não elimina o terceiro excluído
 - Não pode ser duas coisas ao mesmo tempo
 - Compor dois espaços ao mesmo tempo
 - O espaço é a forma
 - Possuir mais de duas formas
 - Equivalentes
 - Contraditórias

Como os atributos e os verbos se relacionam?

Um atributo é uma caixa onde pode se guardar algum estado. Essas caixas se relacionam diretamente com os verbos. A caixa do estado de um objeto aberta ou fechada; ou o estado de um item que pode estar dentro ou fora desta caixa. Assim como o estado aberto ou fechado se relaciona com o verbo abrir e fechar. Bem como dentro ou fora se relaciona com os verbos sair ou entrar. Um verbo pode ser atômico ou composto. O verbo retirar é a composição entre o abrir da caixa e o sair do elemento. Tanto os estados como os verbos possuem relações eternas e imutáveis.

Qual a diferença entre um valor e um objeto?

Um valor é uma caixa singular. Um objeto é uma caixa que possui outras caixas, onde todas elas estão acopladas no mesmo símbolo do objeto.

- Dados são caixas
- Valor é caixa singular / objeto simples
- Objeto é uma caixa com outras caixas / objeto composto
- O verbo deve tratar de qualidade, não de objetos
- A abstração faz o elemento se moldar ao verbo e não o verbo ao elemento, tornando o verbo universal

Objetos devem possuir estados e comportamentos?

- Cada objeto possui sua forma própria de verbo?
 - Exemplo abrir
 - Carta
 - Microondas
 - Porta
 - Provocam o mesmo resultado, mas com execuções diferentes
- Qual a diferença de estados para verbos?
 - Verbos são universais?
 - Estados tbm n são?
 - Ou o verbo não precisa conhecer o objeto?
- Comportamentos são filios
 - Esses filios se encaixam
 - Formam as mesmas histórias

Qual a diferença entre estados e verbos?

- Estados pertencem a verbos?
 - Dois é qualidade ou critério?
 - Dois abstrato é qualidade, pois não há diferença
 - Critério é o espaço?
 - Qualidade é o preenchimento?
 - Dois é espaço?
 - Há diferença de um dois para outro dois?
 - Então é preenchimento?
 - O segredo está na diferenciação?
 - Existem n diferentes números
 - Número passa a ser critério
 - O que é compartilhado é critério?
 - Apenas se houver diferenças?
 - Verbos deixam espaços
 - Tornam-se verbo para a composição
 - Estados pertencem a objetos e objetos pertencem a histórias
 - A programação é a própria linguagem

Por que os estados devem ficar separados do verbo?

- Divisão de responsabilidades e Acoplamento
- Verbos se repetem
 - Qualidades não?
- Verbos não possuem donos
- Verbos mudam qualidades
- Verbos não possuem qualidades
- As qualidades são particulares
 - Os estados são as próprias qualidades
- Os verbos são gerais
- Verbos produzem critérios
- Componentes possuem qualidades
- Os individuais são os elementos sólidos
- A qualidade se encaixa no critério
- Para cada verbo, no mínimo dois estados
- A qualidade é concreta
- A qualidade pertence ao verbo e não o verbo ao objeto
- A história menor colapsa em estado para uma história maior

Qual a diferença entre dado e objeto

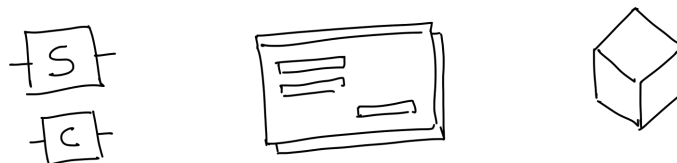
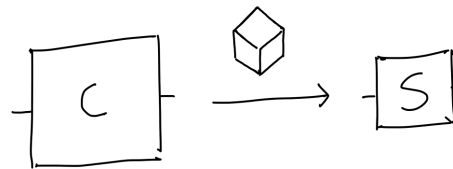
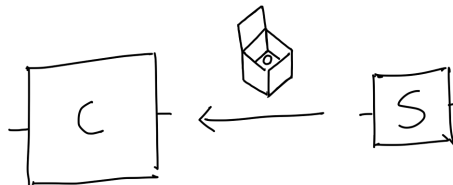
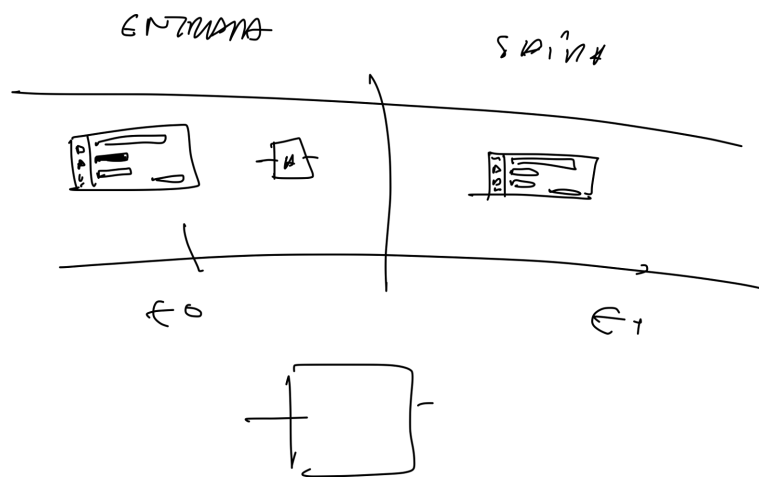
- Atômicos x agregados
- dados são atômicos (chave valor), ou agregados (objetos)
- Substituem em conjunto
- Mais fáceis de portar
- Aplicação possui sujeito e verbo
- Coesão e acoplamento

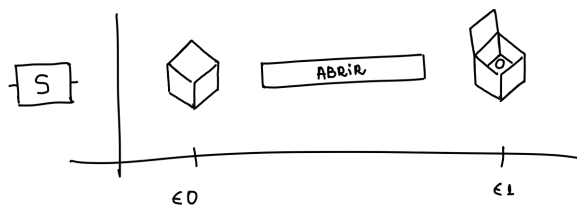
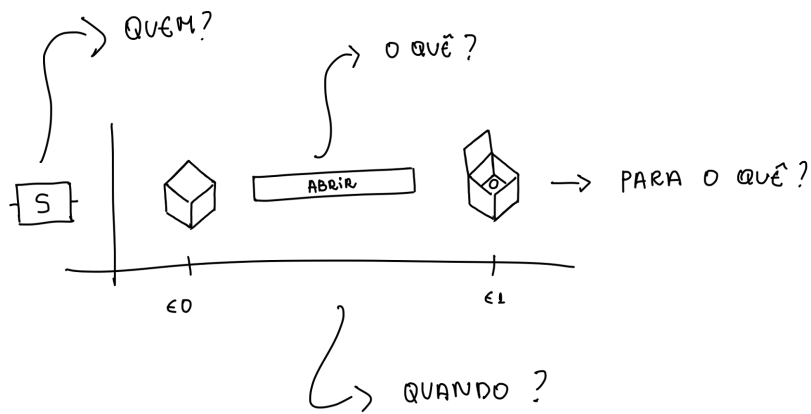
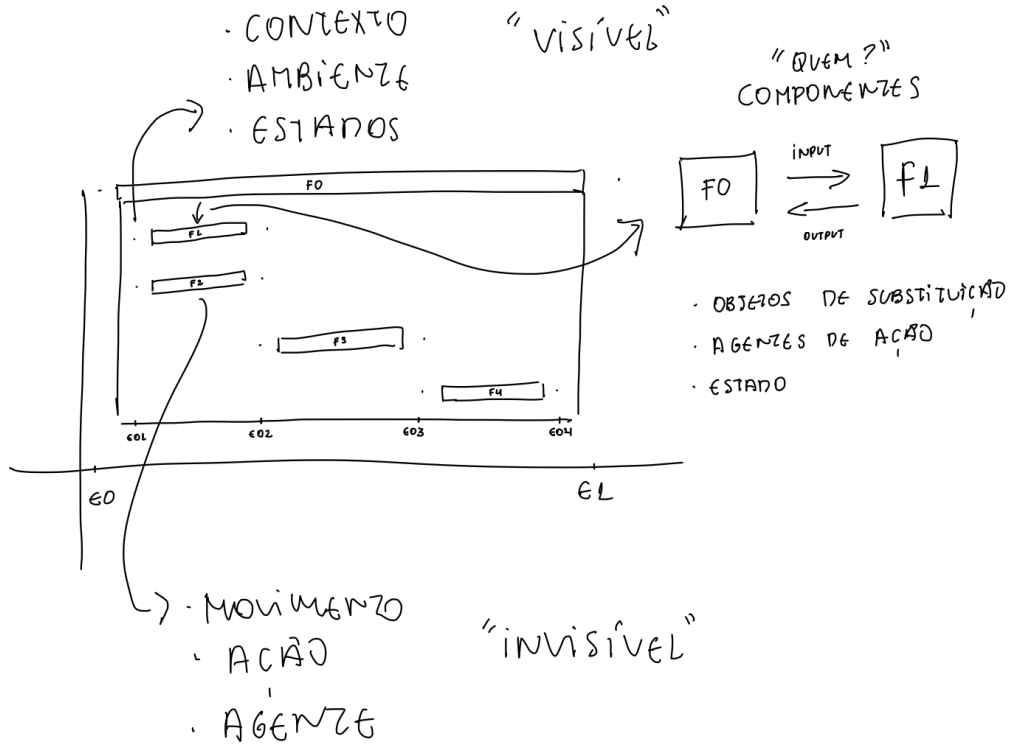
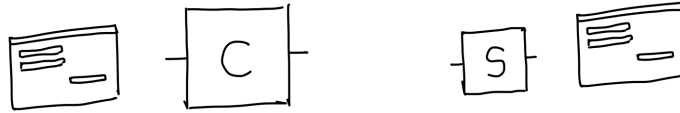
Separar setup de dado?

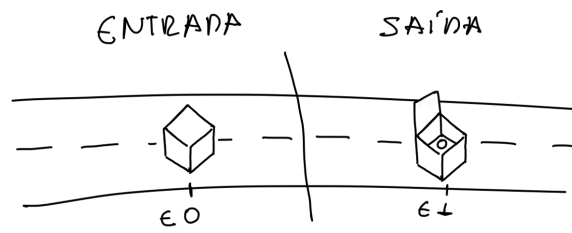
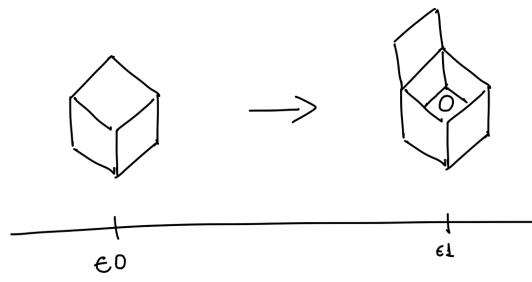
- setup seriam código fonte verbos
- Dado as informações
- setup(verbo)(dado)

Usar dado ou função

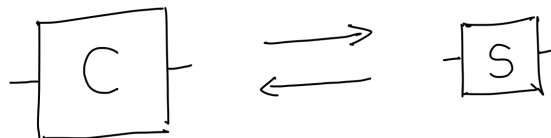
- função tem a capacidade de acessar externamente
 - Dado precisa estar local
 - Talvez deve se seguir o fluxo, dados apenas no início da execução, qualquer outro valor futuro deve vir por resgate de função
 - Talvez de fato separar?
 - setup(verbo, dado)(dado)
 - Desta forma sempre tem um input de dado
 - Qualquer coisa além do input principal é DI
-
- Existe uma relação de fluxo de dados.
 - Dados posteriores ao início devem ser resgatado?
 - Formam componentes
 - Só ter o dado. Deve se utilizar de imediato
 - Posteriormente deve utilizar função

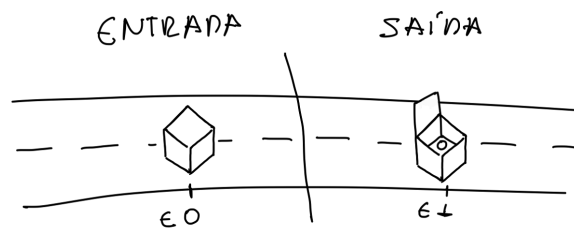




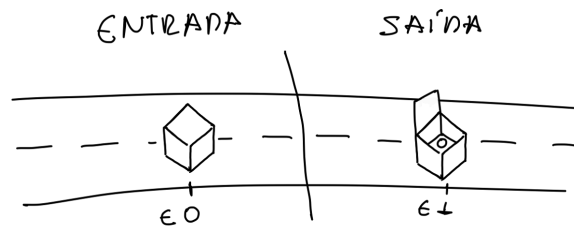


PROCESSAMENTO





PROCESSAMENTO



PROCESSAMENTO

